

Software Requirements Specification (SRS)

Restaurant App MVP (Frontend Only, React Native)

Document Version: 1.0.0

Academic Term: Fall 2026

Course: Mobile Application Development (CS-3112 / SE-3112)

Deliverable: Question 1 — Comprehensive System Specification

Executive Summary & Client Interview

To ground the requirements in real-world human experience, a preliminary client self-interview was conducted from both Customer and Restaurant Manager perspectives:

The Customer Persona: *"I often face tight schedules and long queues during lunch and dinner rushes. When dining out, I need to browse current menu prices and specials, check real-time table availability, and customize my order in advance so my food is prepped when I arrive."*

The Manager Persona: *"Handling telephone orders during peak meal service leads to miscommunication, incorrect notes, and chaotic kitchen queues. We require a digitized visual console to oversee incoming orders, manage table confirmations, and update item availability and pricing instantly without downtime."*

1. Introduction

1.1. Purpose

This Software Requirements Specification (SRS) details the complete functional and non-functional requirements for the **Restaurant App MVP (Gourmet Haven)**. The target audience includes academic evaluators, mobile software engineers, UI/UX designers, and quality assurance testers. This specification strictly reflects a frontend-only architecture powered by React Native and Expo SDK, utilizing local state machines and simulated persistence without a physical backend server.

1.2. Scope

In-Scope Functionalities:

Authentication & Role Authorization: Dual-mode authentication (Customer & Manager) utilizing mock user registries and session storage.

Interactive Menu Catalog: Categorized menu browsing (Starters, Mains, Desserts, Drinks) with simulated network delay, pull-to-refresh, dynamic item counting, and daily special badges.

Optimized Search & Filter Controls: Real-time text search with 400ms debouncing, search history suggestions, category filtering, multi-criteria sorting, and a 300px threshold "Back to top" floating button.

Adaptive Theming: System-wide dark and light mode toggle using custom React Context, providing instant palette transformation.

Cart & Order Customization: Pure reducer-driven cart supporting quantity increments/decrements, item-level dietary instructions, promotional voucher discounts (`WELCOME10` , `FEAST20`), and real-time tab badge indicators.

Financial Calculation & Performance Optimization: Subtotal, service charges (5%), and sales tax (15%) calculated via memoized formulas; food card re-renders optimized via `React.memo` and `useCallback` .

Table Reservation System: Custom hook-based booking pipeline featuring slot conflict detection (12:00 to 22:00), party capacity checks (1–12 guests), Pakistani phone format validation (`03XX-XXXXXXX`), and reservation cancellations.

Live Order Progression & Manager Operations: Dine-in vs. Takeaway checkout, timed status progression (10s Preparing → 20s Ready → 30s Served), live elapsed timer, and a dedicated 3-tab Manager Dashboard for queue management, reservation approvals, and real-time menu CRUD.

Out-of-Scope (Excluded from MVP):

Live Payment Gateway: No real credit card or third-party payment processing (Stripe, JazzCash, EasyPaisa).

Native Push Notifications: Apple APNs or Google FCM remote push notification daemons are excluded.

Remote Backend & Persistent Relational Database: No remote REST/GraphQL microservices, Node.js server, Firebase, or cloud SQL databases.

Hardware Geolocation / GPS Tracking: Real-time courier dispatch and GPS map rendering are excluded.

1.3. Definitions, Acronyms, and Abbreviations

SRS (Software Requirements Specification): A structured document outlining functional capabilities, constraints, and architecture of a software product.

MVP (Minimum Viable Product): A version of a software product with sufficient core features to validate requirements and usability.

UML (Unified Modeling Language): Standardized visual modeling language used to visualize, specify, and document software system artifacts.

React Hook: Special JavaScript functions (e.g., `useState` , `useEffect` , `useRef`) enabling functional components to manage state and lifecycle events.

Context API: React engine feature facilitating global state propagation across deeply nested component trees without manual prop drilling.

Pure Reducer: A deterministic function `(state, action) => newState` that calculates subsequent application state without mutating original inputs or causing side effects.

Mock Data: Localized JSON or JavaScript data structures simulating realistic database entities for standalone frontend development.

FlatList: A memory-efficient React Native component that virtualizes and renders only currently visible items within large scrollable datasets.

Debounce: A software design technique that restricts the execution of a computationally expensive task until a specified duration of inactivity has elapsed.

Immutable State: An architectural paradigm where data objects cannot be modified post-creation; state transitions always yield fresh object references.

2. Overall Description

2.1. User Roles & Personas

Role	Description	Key Permissions & Goals
Customer (Diner)	Registered or guest patron seeking culinary browsing, ordering, and dining reservations.	• Browse categorized menu items and daily specials. • Search with instant debouncing and sorting. • Customize cart items with chef instructions. • Apply promotional discount vouchers. • Book and cancel table reservations. • Place Dine-in / Takeaway orders and monitor live kitchen progress. • Toggle dark/light luxury UI themes.
Restaurant Manager (Staff)	Administrative culinary supervisor overseeing restaurant floor and kitchen throughput.	• Access the secured Manager Operations Console. • Monitor live incoming kitchen orders and advance status stages manually. • Review pending table reservations, accepting or declining bookings. • Update item prices, toggle dish availability, and introduce new menu items with instant sync across customer screens.

2.2. User Stories

Customer User Stories (Derived from Client Meeting / Self-Interview):

US-01 (Menu Discovery): *As a Customer,* I want to view dishes categorized into Starters, Mains, Desserts, and Drinks with appetizing images and prices, so that I can quickly decide what to eat.

US-02 (Daily Specials): *As a Customer*, I want chef specials clearly highlighted with distinctive badges, so that I can discover unique seasonal recommendations.

US-03 (Instant Search): *As a Customer*, I want a fast, debounced search bar with recent search history chips, so that I can find my favorite meals without re-typing.

US-04 (Dietary Customization): *As a Customer*, I want to attach per-item special instructions (e.g., "no onions", "extra spicy") to dishes in my cart, so that the kitchen prepares my food according to my preferences.

US-05 (Promotional Vouchers): *As a Customer*, I want to enter promo codes (such as `WELCOME10` or `FEAST20`) during checkout, so that I receive discounts on my total bill.

US-06 (Advance Table Booking): *As a Customer*, I want to reserve a table for a specific date, hourly slot, and party size with automated capacity validation, so that I don't face waiting lines upon arrival.

US-07 (Live Order Progress): *As a Customer*, I want to track my order through an automated progress timeline with elapsed seconds, so that I know exactly when my food is preparing, ready, or served.

Restaurant Manager User Stories:

US-08 (Kitchen Queue Progression): *As a Restaurant Manager*, I want a real-time incoming orders dashboard where I can advance kitchen tickets from Pending to Preparing, Ready, and Served, so that order delivery remains synchronized.

US-09 (Reservation Management): *As a Restaurant Manager*, I want to review incoming dining reservations and explicitly Accept or Decline them based on restaurant floor capacity, so that tables are never overbooked.

US-10 (Live Menu CRUD & Availability): *As a Restaurant Manager*, I want to adjust menu item prices and toggle out-of-stock items off in real-time, so that customers cannot order unavailable ingredients.

US-11 (Role-Restricted Console): *As a Restaurant Manager*, I want management tabs to be completely hidden from ordinary diner profiles, so that sensitive administrative actions remain secure.

US-12 (Theme Personalization): *As a Customer or Manager*, I want to toggle between obsidian dark and warm amber light themes, so that I can comfortably use the app in varying ambient lighting.

3. Functional Requirements

Module 1: Authentication & Authorization

FR-01: The system shall toggle between Login and Signup modes using a controlled mode state variable on a single interactive screen.

FR-02: The system shall validate email syntax, enforce password complexity (minimum 8 characters with at least one numerical digit), and confirm matching passwords prior to submission.

FR-03: The system shall clear validation error indicators immediately when a user modifies the text within an erroneous input field.

FR-04: The system shall authenticate user credentials against a local mock user registry, simulating a 1000ms network delay with an active loading spinner and disabled submit button.

FR-05: The system shall route authenticated Customer users to the main Menu screen and Manager users directly to the Manager Dashboard.

Module 2: Menu Browsing & Catalog

FR-06: The system shall asynchronously load at least 15 menu items spanning 4 distinct categories (Starters, Mains, Desserts, Drinks) using a simulated 1.5-second Promise with unmount cancellation.

FR-07: The system shall present horizontal category filtering chips allowing patrons to view items by specific course or all items simultaneously.

FR-08: The system shall render catalog items in an optimized `FlatList`, displaying distinct "Daily Special" badges and greying out unavailable dishes with disabled action buttons.

FR-09: The system shall implement pull-to-refresh functionality, re-executing the menu fetch workflow and updating the total displayed item count in the header.

Module 3: Search, Filter & Scroll Optimization

FR-10: The system shall provide an interactive search bar with a persistent `useRef` focus trigger and a 1-tap clear button that resets input without losing cursor focus.

FR-11: The system shall debounce search queries by 400 milliseconds, eliminating redundant filtering calculations during active typing.

FR-12: The system shall present the last five unique search queries as tappable suggestions when the search bar is focused.

FR-13: The system shall display a floating "Back to top" button when the list is scrolled past 300 vertical pixels, invoking `scrollToOffset({ offset: 0, animated: true })`.

FR-14: The system shall display a friendly empty state message whenever no catalog items match active search queries or category filters.

Module 4: Shopping Cart & Reducer State

FR-15: The system shall manage cart operations through a pure `cartReducer` handling `ADD_ITEM`, `REMOVE_ITEM`, `INCREMENT`, `DECREMENT` (auto-removing at quantity zero), `UPDATE_NOTE`, `CLEAR_CART`, `APPLY_PROMO`, and `REMOVE_PROMO`.

FR-16: The system shall validate promotional discount vouchers (`WELCOME10` for 10% off, `FEAST20` for 20% off), rejecting unlisted codes with informative error notifications.

FR-17: The system shall update a live badge indicator on the bottom navigation tab icon reflecting the total cumulative item count in the tray.

Module 5: Order Summary & Financial Calculation

FR-18: The system shall compute Subtotal, Service Charge (5%), Provincial Sales Tax (15%), Promotional Discounts, and Grand Total using pure memoized expressions derived from cart items.

FR-19: The system shall support both Dine-in (with table assignment) and Takeaway (with designated pickup time) dining modes.

Module 6: Table Reservation Pipeline

FR-20: The system shall provide hourly reservation slots between 12:00 and 22:00, disabling slots when restaurant capacity is exhausted.

FR-21: The system shall validate reservation parameters, ensuring dates are not in the past, party sizes range from 1 to 12 guests, lead time is at least 1 hour, and contact numbers conform to Pakistani mobile formatting (`03XX-XXXXXXX`).

FR-22: The system shall present a booking confirmation modal prior to committing a reservation, and permit patrons to cancel existing reservations with an Alert dialog confirmation.

Module 7: Live Order Tracking & Manager Dashboard

FR-23: The system shall provide an Order Tracking screen that automatically transitions ticket status from `Pending` → `Preparing` (10s) → `Ready` (20s) → `Served` (30s) while rendering a real-time elapsed counter that updates every second.

FR-24: The system shall provide a Manager Dashboard featuring three dedicated operational views: (a) Incoming Orders queue with manual status overrides, (b) Table Reservation approvals/declines, and (c) Live Menu Management (item creation, price edits, availability toggles).

FR-25: The system shall synchronize menu modifications and order status transitions instantly across Manager and Customer views via shared Context providers and persistent local storage.

4. Non-Functional Requirements

NFR-01 (Usability & Aesthetics): The application shall follow luxury dining design aesthetics, using custom warm culinary amber and obsidian color palettes, 16px touch targets, smooth 3D button animations, and high-contrast typography.

NFR-02 (Performance & 60 FPS Scrolling): Heavy list cards (`MenuItemCard`) shall be memoized using `React.memo` and stable `useCallback` action references. Toggling an item's favorite state shall re-render only the target card.

NFR-03 (Responsiveness): UI layouts shall dynamically adapt across standard mobile phone viewports (360px to 480px width) and tablet form factors, utilizing flexible percentages, `Dimensions` listeners, and `SafeAreaProvider` .

NFR-04 (Maintainability & Clean Architecture): Source code shall strictly adhere to modular separation under `src/` (components, screens, context, reducers, hooks, data, navigation, theme).

NFR-05 (Client-Side Data Integrity): The application shall execute without external server dependencies, persisting key user sessions, custom menu items, orders, and reservations across app restarts via `safeStorage` (in-memory + AsyncStorage fallback).

NFR-06 (Memory Leak Prevention): Every asynchronous timer (`setTimeout` , `setInterval`) across loading screens and order tracking steppers shall include explicit cleanup functions executed upon component unmount.

5. Client-Side Data Model (Mock Data Schema)

Data Set Name	Primary Fields & Data Types	Description & Business Purpose		
Users	<code>id</code> (String), <code>name</code> (String), <code>email</code> (String), <code>password</code> (String), <code>role</code> ('customer' \	'manager'), <code>phone</code> (String), <code>avatar</code> (String)	Stores patron and restaurant staff profiles used for authentication and role-based routing.	
Categories	<code>id</code> (String), <code>name</code> (String), <code>icon</code> (String)	Enumerates the 4 culinary courses: Starters, Mains, Desserts, and Drinks.		
MenuItems	<code>id</code> (String), <code>name</code> (String), <code>description</code> (String), <code>price</code> (Number), <code>category</code> (String), <code>image</code> (String), <code>isSpecial</code> (Boolean), <code>isAvailable</code> (Boolean), <code>prepTime</code> (String), <code>rating</code> (Number)	Catalog of gourmet dishes available for order; editable in real-time by kitchen managers.		
Tables	<code>id</code> (String), <code>number</code> (Number), <code>capacity</code> (Number), <code>location</code> ('Indoor' \	'Patio' \	'Rooftop'), <code>isOccupied</code> (Boolean)	Physical dining tables used for seating assignment and party size matching.

Data Set Name	Primary Fields & Data Types	Description & Business Purpose							
Reservations	id (String), customerName (String), customerPhone (String), partySize (Number), date (String), timeSlot (String), tableId (String), status ('confirmed' \	'pending' \	'declined' \	'cancelled'),	createdAt (String)	Table bookings made by customers; reviewed and actioned by restaurant managers.			
Orders	id (String), userId (String), customerName (String), items (Array of CartItem), subtotal (Number), serviceCharge (Number), tax (Number), discount (Number), total (Number), orderType ('dine-in' \	'takeaway'),	tableNumber (Number/null), status ('Pending' \	'Preparing' \	'Ready' \	'Served' \	'Cancelled'),	timestamp (String)	Active culinary orders dispatched from checkout, tracked live by patrons and updated by staff.

6. UML Architectural Diagrams & Summaries

6.1. Use Case Diagram

Caption: Figure 6.1 — Gourmet Haven Use Case Diagram (Actors: Customer & Restaurant Manager).

Architectural Explanation: The use case diagram models interactions between the primary actors (Customer and Restaurant Manager) and the system boundaries. The Customer interacts with 8 core use cases including Menu Discovery, Search & Filter, Cart Customization, Table Reservation, and Order Placement. The Place Order

use case demonstrates an `<>` relationship with `Calculate Totals` and an `<>` relationship with `Apply Promo Code`. The Restaurant Manager oversees kitchen workflows through `Manage Incoming Orders`, `Review Reservations`, and `Update Menu Catalog`.

6.2. Class / Domain Model Diagram

Caption: Figure 6.2 — Object-Oriented Domain Model & Class Relationships.

Architectural Explanation: The domain model represents system entities and structural multiplicities. The base class `User` is specialized into `Customer` and `Manager`. A `Customer` aggregates one `Cart`, which holds 0..* `CartItem` instances referencing `MenuItem` objects. An `Order` is generated from a `Cart`, encapsulating financial computations and referencing an optional `Table`. Tables maintain a 1-to-many relationship with `Reservations`. All methods adhere to encapsulation and type safety.

6.3. Sequence Diagram (Customer Places an Order)

Caption: Figure 6.3 — End-to-End Sequence Diagram for Order Placement.

Architectural Explanation: Traces the chronological message exchanges between `Customer`, `MenuScreen`, `CartScreen`, `cartReducer`, `OrdersContext`, and `OrderTrackingScreen`. When a diner taps "Add to Cart", a dispatch invokes `ADD_ITEM`. In `OrderSummaryScreen`, the diner selects dining mode and confirms checkout. `OrdersContext` instantiates an order record, clears the cart via `CLEAR_CART`, and navigates the patron to `OrderTrackingScreen` where interval timers initiate live status progression.

6.4. State Machine Diagram (Order Lifecycle)

Caption: Figure 6.4 — Finite State Machine for Culinary Order Lifecycle.

Architectural Explanation: Illustrates the discrete lifecycle states of an order: `Pending` → `Preparing` → `Ready` → `Served`. Secondary branch transitions include cancellation to `Cancelled` from `Pending` state. Transitions are triggered either by automated interval timers (10s, 20s, 30s) or manual manager status override events from the Operations Console.

6.5. Component Diagram (React Native Architecture)

Caption: Figure 6.5 — React Native Component & Context Dependency Diagram.

Architectural Explanation: Shows the modular frontend architecture. The root `AppNavigator` encapsulates `SafeAreaProvider` and 5 Context Providers (`ThemeProvider`, `AuthProvider`, `MenuProvider`, `CartProvider`, `OrdersProvider`). Screens consume contexts via specialized custom hooks (`useTheme`, `useAuth`, `useCart`, `useOrders`, `useReservation`). Subordinate presentation components such as `MenuItemCard` are decoupled and optimized with `React.memo`.

7. MVP Frontend Development (React Native Screen Specifications)

7.1. Login and Signup Screen (`LoginScreen.js`)

Purpose: Single-screen authentication portal with 3D animated flip transitions for login and registration.

UI Elements: Mode toggle pills, controlled text inputs for Name, Email, Password, Confirm Password, eye password toggles, 3D submit button, quick-fill demo profile chips.

Navigation Entry & Exit: Entry point of the app; on customer login/signup navigates to `CustomerApp (Menu)`, on manager login navigates to `ManagerDashboard`.

Local Data & State: `mode` ('login' | 'signup'), `name`, `email`, `password`, `confirmPassword`, `errors` object, `showPassword`, `isSubmitting`.

Hooks Used & Why: `useState` for controlled inputs and errors; `useRef` for 3D bounce scale animation; `useAuth` to dispatch credentials; `useTheme` for dynamic styling.

7.2. Menu Browsing Screen (`MenuScreen.js`)

Purpose: High-end culinary menu showcase featuring real-time category filtering and special highlights.

UI Elements: Horizontal category selector chips, pull-to-refresh FlatList, dish cards with image, price, rating, prep time, "Daily Special" badge, and heart favorite toggle.

Navigation Entry & Exit: Default tab in `CustomerTabs`; can navigate to item details or tray.

Local Data & State: `menuItems`, `isLoading`, `error`, `refreshing`, `selectedCategory`, `favouriteIds` array.

Hooks Used & Why: `useState` for items and filters; `useEffect` for 1.5s simulated Promise fetch with unmount cleanup; `useMemo` for derived category lists; `useCallback` for memoized item handlers.

7.3. Search and Scroll Controls (Integration in `MenuScreen.js`)

Purpose: High-speed item search with debouncing and instant scroll-to-top controls.

UI Elements: Search input bar with search icon, clear button, recent search suggestion tags, floating "Back to top" arrow button, render counter debug label.

Navigation Entry & Exit: Embedded directly within the Menu screen view hierarchy.

Local Data & State: `searchQuery`, `debouncedQuery`, `isSearchFocused`, `recentSearches` array, `showBackToTop` boolean.

Hooks Used & Why: `useRef` to maintain TextInput reference and persistent render counter; `useDebounce` custom hook for 400ms delay; `useRef` for FlatList scrollToOffset; `useMemo` for combined filter and sort pipeline.

7.4. Profile and Theme Screen (`ProfileScreen.js`)

Purpose: Patron profile overview and global dark/light luxury theme configuration.

UI Elements: User avatar, name, email, role badge, theme toggle switch with sun/moon icons, demo credentials helper, logout button.

Navigation Entry & Exit: Fourth tab in `CustomerTabs` ; logout resets navigation stack back to `Login` .

Local Data & State: Reads active session from `useAuth` and theme state from `useTheme` .

Hooks Used & Why: `useAuth` for user details and `logout()` ; `useTheme` for `isDark` and `toggleTheme()` .

7.5. Cart Screen (`CartScreen.js`)

Purpose: Interactive culinary tray allowing quantity adjustments, dietary notes, and voucher redemption.

UI Elements: Scrollable list of cart items with quantity steppers (+/-), per-item special instruction input fields, delete button, promo code entry with apply button, checkout button.

Navigation Entry & Exit: Second tab in `CustomerTabs` ; checkout button navigates to `OrderSummary` .

Local Data & State: `promoInput` , `promoError` , `promoSuccess` , items from `useCart` .

Hooks Used & Why: `useCart` to access reducer state and dispatch actions (`INCREMENT` , `DECREMENT` , `UPDATE_NOTE` , `APPLY_PROMO`); `useTheme` for theme tokens.

7.6. Order Summary Screen (`OrderSummaryScreen.js`)

Purpose: Itemized financial verification and dining mode selection prior to kitchen dispatch.

UI Elements: Dine-in vs. Takeaway selector tabs, Table number selector / pickup time picker, itemized breakdown, tax/service charge summary card, confirm order button.

Navigation Entry & Exit: Pushed onto RootStack from `CartScreen` ; on confirmation navigates to `OrderTracking` .

Local Data & State: `diningMode` ('dine-in' \| 'takeaway'), `selectedTable` , `pickupTime` .

Hooks Used & Why: `useMemo` for subtotal, 5% service charge, 15% tax, discount, and grand total; `useOrders` to dispatch `createOrder()` ; `useCart` to trigger `clearCart()` .

7.7. Table Reservation Screen (`ReservationScreen.js`)

Purpose: Dine-in table booking portal with real-time slot conflict and seating capacity validation.

UI Elements: Date picker chips, hourly time slot pills (12:00–22:00) with disabled unavailable states, party size stepper (1–12), contact phone input, confirmation modal, My Reservations list.

Navigation Entry & Exit: Third tab in `CustomerTabs` .

Local Data & State: Pure UI presentation; delegates all state management to custom hook.

Hooks Used & Why: `useReservation` custom hook for all business logic (selected slot, party size, availability calculation, booking creation, and cancellations); `useTheme` for styling.

7.8. Order Tracking & Manager Dashboard

Purpose: Real-time customer kitchen progress monitor and comprehensive manager operations console.

UI Elements:

Tracking:* Status step progress bar (`Pending` → `Preparing` → `Ready` → `Served`), running elapsed seconds timer counter, order items receipt.

Manager:* 3-Tab interface — (1) Incoming Orders with manual status step buttons, (2) Reservation requests with Accept/Decline, (3) Menu item manager with price editing, availability toggles, and new item creation.

Navigation Entry & Exit: `OrderTracking` is pushed from `OrderSummary` ; `ManagerDashboard` is accessed upon manager login or via restricted tab.

Local Data & State: `activeTab` ('orders' \| 'reservations' \| 'menu'), `elapsedSeconds` , `editModalVisible` .

Hooks Used & Why: `useEffect` with `setInterval` for automated status progression and 1-second elapsed counter; unmount cleanup functions; `useOrders` , `useMenu` , and `useReservation` for shared synchronized state.

End of Software Requirements Specification (SRS)